Федеральное государственное автономное образовательное учреждение высшего образования
«РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ ИМЕНИ ПАТРИСА ЛУМУМБЫ»
(РУДН)

Основное учебное подразделение факультет физико-математических и естественных наук

Направление/специальность 02.03.01 Математика и компьютерные науки (профиль «Математика и компьютерные науки»)

ОТЧЕТ

по научно-исследовательской работе
«научно-исследовательская работа»
(вид и наименование практики)

_____

(Ф.И.О. обучающегося)

Курс, группа 4, НКНбд-0-22

Сроки прохождения с 04.03.2026 г. по 4.04.2026 г.

Руководители практики:

от РУДН Солпагаров Солтан Исмаилович
Ф.И.О., должность

от организации (предприятия)
Шорохов Сергей Геннадьевич, к.ф.м. наук, доцент ИКНиТ
Ф.И.О., должность

Оценка_____

Москва 2026 г.

Оглавление

Раздел	**Страница начала**
Аннотация RU	1
Аннотация EN	2
Список русскоязычных сокращений	3
Список англоязычных сокращений	4
Введение	8
Обзор	10
Описание разработки	12
Реализация	17
Тестирование и апробация	21
Заключение	26
Литература	27

Аннотация

В работе рассматривается задача оценки американских опционов пут в рамках дискретной биномиальной модели Кокса–Росса–Рубинштейна. Актуальность связана с потребностью в быстрых и интерпретируемых численных методах ценообразования деривативов и с развитием гибридных подходов, сочетающих классические финансово-математические модели с аппаратом глубокого обучения. Целью научно-исследовательской работы является программная реализация и численная проверка нейросетевой архитектуры, эквивалентной обратному ходу по биномиальному дереву, включая вариант с maxout-слоями для учёта возможности досрочного исполнения американского опциона. В качестве инструментов использованы язык Python и библиотека PyTorch; эталонные значения для европейских опционов получены по формуле Блэка–Шоулза, для американских — по биномиальной схеме с увеличенным числом шагов; результаты нейросети сопоставляются также с ценами на дереве с фиксированным числом периодов. Проведено обучение сетей методом стохастического градиента с минимизацией среднеквадратичной ошибки на синтетических выборках по цене исполнения. Установлено, что обученная модель согласуется с биномиальным эталоном соответствующей глубины; расхождения с аналитической моделью Блэка–Шоулза и с «тонкой» биномиальной сеткой интерпретируются как следствие дискретизации времени и специфики американского контракта. Результаты подтверждают применимость предложенной архитектуры для аппроксимации цен в заданной параметризации рынка и задают основу для дальнейшего развития модели на более сложных предположениях о динамике базового актива.

Abstract

The thesis addresses pricing of American put options within the discrete Cox–Ross–Rubinstein binomial framework. The topic is motivated by the need for efficient, interpretable numerical tools for derivative valuation and by hybrid approaches that combine classical financial models with deep learning. The goal of the research is to implement and numerically validate a neural architecture equivalent to backward induction on a binomial tree, including a maxout-based variant that encodes early-exercise features of American options. Implementation relies on Python and PyTorch. European option prices from the Black–Scholes formula serve as training targets in the baseline experiment, while American targets are produced with a finer binomial tree; network outputs are compared with both the coarse tree used in the architecture and these reference values. Models are trained by stochastic gradient descent minimizing mean squared error on synthetic samples over strike prices. Empirically, the trained networks align with the binomial reference of matching depth, whereas deviations from Black–Scholes prices and from a high-resolution tree reflect time discretization and American exercise policy. The findings support using the proposed architecture to approximate option prices under the stated market parameters and outline directions for extensions such as richer volatility structures or market-calibrated training.

Список сокращений

Русскоязычные сокращения

Сокращение	Расшифровка
ВКР	Выпускная квалификационная работа
НИР	Научно-исследовательская работа
ПК	Программный комплекс
РУДН	Российский университет дружбы народов
ТУИС	Телекоммуникационная учебно-информационная система
ФОС	Фонд оценочных средств

Англоязычные сокращения

Сокращение	Полное название	Перевод
Adam	Adaptive Moment Estimation	Адаптивная оценка моментов
BS	Black-Scholes	Блэк-Шоулз (модель) [3]
BT	Binomial Tree	Биномиальное дерево
CRR	Cox-Ross-Rubinstein	Кокс-Росс-Рубинштейн (модель)
1D	One-Dimensional	Одномерный
IBT	Implied Binomial Tree	Имплицитное биномиальное дерево
ML	Machine Learning	Машинное обучение
MSE	Mean Squared Error	Среднеквадратичная ошибка
NN	Neural Network	Нейронная сеть
PDF	Probability Density Function	Функция плотности вероятности
ReLU	Rectified Linear Unit	Выпрямленная линейная единица (функция активации)

Сокращение	Полное название	Перевод
PyTorch	-	Фреймворк для машинного обучения (Meta)

Обозначение	Описание	Единица измерения
S	Цена базисного актива	Денежные единицы
S_0	Текущая цена базисного актива (в момент t_0)	Денежные единицы
S_j^i	Цена базисного актива в узле (j) биномиального дерева	Денежные единицы
K	Цена-страйк (цена исполнения) опциона	Денежные единицы
T	Время погашения (исполнения) опциона	Годы
t_0	Текущий момент времени	Годы
$\tau = T - t_0$	Срок до истечения опциона	Годы
n	Количество периодов в биномиальном дереве	-

Обозначение	Описание	Единица измерения
$\Delta t = \tau/n$	Длина одного периода	Годы
r	Безрисковая процентная ставка	% в год
σ	Волатильность базисного актива	%
u	Множитель роста цены базисного актива	-
d	Множитель падения цены базисного актива	-
π_u	(Псевдо)вероятность роста цены базисного актива	-
π_d	(Псевдо)вероятность падения цены базисного актива	-
V_j^i	Стоимость европейского опциона в узле (j)	Денежные единицы
$\tilde{V}_j^i$	Стоимость американского опциона в узле (j)	Денежные единицы
w^i	Вектор весов i-го слоя нейронной сети	-
b^i	Вектор смещений i-го слоя нейронной сети	-

Обозначение	Описание	Единица измерения
W	Фильтр сверточного слоя (размер 2)	-
$ReLU(x)$	Функция активации ReLU: $ma(x, 0)$	-
$maxout$	Функция активации, выбирающая максимум из двух входов	-
$softmax$	Функция активации для многоклассовой классификации	-

1. Введение

Актуальность. Рынок деривативов опирается на корректную оценку опционов. Для европейских контрактов в модели Блэка–Шоулза–Мертона известны аналитические соотношения [3], [4]. Для **американских** опционов, допускающих досрочное исполнение, в той же постановке, как правило, **нет универсальной формулы в замкнутом виде**; применяются решёточные методы, схемы для уравнений в частных производных с подвижной границей, методы Монте-Карло с оценкой условного ожидания [8], [9]. Параллельно развиваются **нейросетевые** методы ценообразования и вычисления подразумеваемой волатильности [7]. Отдельный интерес представляют **структурированные** архитектуры, где слои сети соответствуют шагам известного дискретного алгоритма: так сохраняется интерпретируемость и связь с классической теорией [1], [4], [5].

Цель работы — реализовать нейросетевые модели **BTNetEuropean** и **BTNetAmerican**, соответствующие биномиальной схеме CRR [2], провести обучение на синтетических данных и **верифицировать** результаты путём сравнения с эталонами: аналитика Блэка–Шоулза для европейского пута [3], биномиальное дерево с заданным числом шагов [2], [4], [5] и **густое** биномиальное дерево для американского пута; при необходимости сопоставление с библиотекой **QuantLib** [10] (в соответствии с индивидуальным заданием на НИР).

Задачи исследования:

1. Проанализировать классические и нейросетевые подходы к оценке опционов (CRR, BS, численные методы, работы по ML в финансах) и выделить место структурированной архитектуры [1], [2], [7].

2. Описать **математическую** модель биномиального дерева и отображение обратного хода на архитектуру сети (полносвязный слой, свёртки, maxout) с указанием условий согласования весов с параметрами CRR [1], [2], [5], [6].

3. Реализовать программный комплекс на **Python** с использованием **PyTorch** [11], вспомогательных библиотек **NumPy**, **SciPy** [12], **Matplotlib** [13], зафиксировать зависимости и оформить воспроизводимый эксперимент в ноутбуке.

4. Провести **численные эксперименты**, построить таблицы и графики, оценить метрики отклонения от эталонов и сформулировать выводы о работоспособности архитектуры.

Объект исследования — процессы и методы оценки опционов пут в дискретной биномиальной постановке. **Предмет исследования** — нейросетевая архитектура BTNet и её программная реализация с анализом точности.

Трек выполнения НИР: П (прикладной) — разработка программного комплекса, документации и результатов вычислительной апробации.

2. Обзор

Биномиальная модель CRR. Модель Кокса–Росса–Рубинштейна [2] дискретизирует время и цену базового актива; цена опциона получается **обратным ходом** по дереву с риск-нейтральными вероятностями. Для американского опциона на каждом узле выполняется сравнение ценности удержания и немедленного исполнения [4], [5], [8]. Метод нагляден и широко используется в обучении и в инженерных расчётах; точность растёт с числом шагов за счёт вычислительных затрат.

Модель Блэка–Шоулза и европейский опцион. В непрерывном пределе при постоянной волатильности цена **европейского** пута задаётся аналитически [3]; этот результат служит **независимым эталоном** при проверке европейской ветви эксперимента. Расхождения дискретной схемы с n шагами и непрерывной модели интерпретируются как **ошибка дискретизации** [4], [8].

Численные методы для американских опционов. Помимо деревьев, применяются конечно-разностные схемы для соответствующих задач со свободной границей [8], метод **Longstaff–Schwartz** (регрессия по симулированным траекториям) [9], методы Монте-Карло [8]. Эти подходы конкурируют с решёточными по точности и масштабируемости в многомерном случае, но **не совпадают по форме** с фиксированной глубиной дерева n, используемой в BTNet.

Нейросети в ценообразовании опционов. Обзорные и прикладные работы демонстрируют использование нейронных сетей для аппроксимации цен и решений уравнений ценообразования [7]. Типичный подход — обучение на

больших выборках признаков; минус — слабая связь слоёв с конкретным дискретным алгоритмом.

Работа Шорохова С. Г. (2024). В [1] показано, как **обратный ход** по биномиальному дереву для европейского опциона реализуется как прямой проход по сети из полносвязного слоя с ReLU и последовательности **одномерных свёрток** с ядром размера 2; для **американского** пута шаги обобщаются с помощью операций **max** между ветвью дисконтирования и ветвью от страйка (maxout-подобная структура). При специальном выборе весов выход сети **совпадает** с ценой по CRR; далее веса могут **дообучаться** под данные. Настоящая работа опирается на эту постановку и реализует её в среде PyTorch [11].

Вывод по обзору. Классика (CRR, BS, MC, PDE) задаёт эталоны и контекст; нейросети [7] дают гибкость; подход [1] выбран как **основа разработки** за счёт явной связи с биномиальным деревом и интерпретируемости весов.

3. Описание разработки

3.1. Дискретизация CRR и вектор дисконтирования

В ноутбуке **btnn-bs-torch.ipynb** параметры дерева задаются так же, как в классической схеме Кокса–Росса–Рубинштейна [2], [4], [5]: для числа периодов n (в коде — аргумент n_dim) вводится шаг $\Delta t = (T - t_0)/n$, множители $u = \exp(\sigma\sqrt{\Delta t})$, $d = \exp(-\sigma\sqrt{\Delta t})$. Дисконтированное риск-нейтральное усреднение «соседних» узлов кодируется **двумя коэффициентами**, собранными в массив W длины 2:

- первый коэффициент: $(\exp(-r\Delta t)\cdot u - 1) / (u - d)$;
- второй: $(1 - \exp(-r\Delta t)\cdot d) / (u - d)$.

Именно этот вектор подаётся в свёрточные слои как вес nn.Conv1d с kernel_size=2 и bias=False [1], [11]. Таким образом, каждый шаг одномерной свёртки по вектору ценностей соответствует одному шагу обратной индукции по биномиальному слою времени.

3.2. Европейская сеть BTNetEuropean

Вход модели — скалярный или батчевый страйк K, приводимый во forward к тензору размерности ($batch$, 1) и типа float.

Первый слой — экземпляр DenseLayer с input_dim=1, output_dim=n_dim+1, активацией torch.relu. Матрица весов инициализируется единичным вектором-строкой размера (1, n+1); смещения задаются как $-S_0\cdot\exp(\sigma\sqrt{\Delta t}\cdot(2j - n))$ для $j = 0, \ldots, n$ — это согласуется с постановкой [1] о представлении терминального слоя дерева через ReLU от линейной формы от K.

Каскад свёрток. Важная особенность реализации: создаётся **один** объект ConvLayer с фиксированным фильтром W, и в цикле for i in range(self._n_dim) он **последовательно применяется *n* раз** к активации. То есть веса свёртки **разделяются** между всеми шагами обратного хода (аналогично тому, что на каждом горизонтальном шаге дерева используется одно и то же правило дисконтирования). После каждой свёртки длина вектора уменьшается на 1: с *n*+1 до 1. На последнем шаге результат приводится к скаляру на объект обучения: если размерность (*batch*, 1), выполняется squeeze; иначе используется усреднение по лишней размерности — это технический приём согласования формы тензора с вектором целевых цен при обучении.

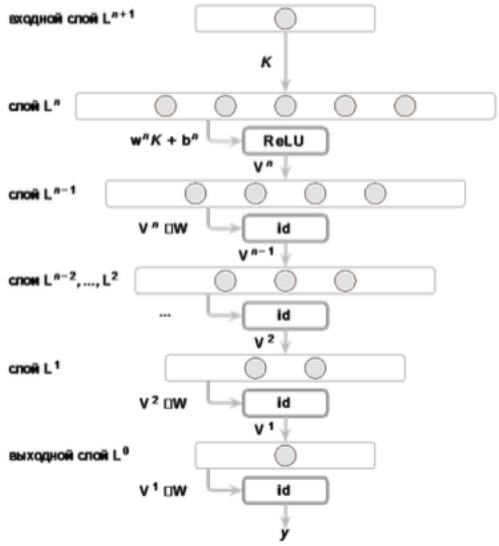

Рисунок 1. Архитектура нейронной сети для оценки европейского опциона пут

3.3. Американская сеть BTNetAmerican

Первый слой полностью аналогичен европейскому варианту (тот же DenseLayer с ReLU и теми же формулами *W_init*, *b_init*).

Далее в nn.ModuleList добавляется **ровно *n* модулей** MaxoutLayer. Для шага с индексом *i* задаются размерности: input_dim = n_dim + 1 - i, output_dim =

n_dim - i. Линейная ветвь maxout инициализируется весами из единиц и смещениями $-S_0 \cdot \exp(\sigma\sqrt{\Delta t}\cdot(2j - i))$, $j = 0, \ldots,$ output_dim-1, то есть **для каждого временного слоя** своя пара (*W_linear*, *bias*), как требует американская рекурсия с внутренней стоимостью исполнения в каждом узле [1], [4].

В MaxoutLayer.forward(V_prev, K) ветвь «удержания» получается так: V_prev приводится к форме (*batch*, 1, *L*), применяется тот же Conv1d с ядром из W, затем снова squeeze; ветвь «исполнения» — это nn.Linear(1, output_dim) от **исходного** *K* (по батчу). Поэлементный torch.max двух векторов реализует условие американского пута: max(цена при удержании, внутренняя стоимость).

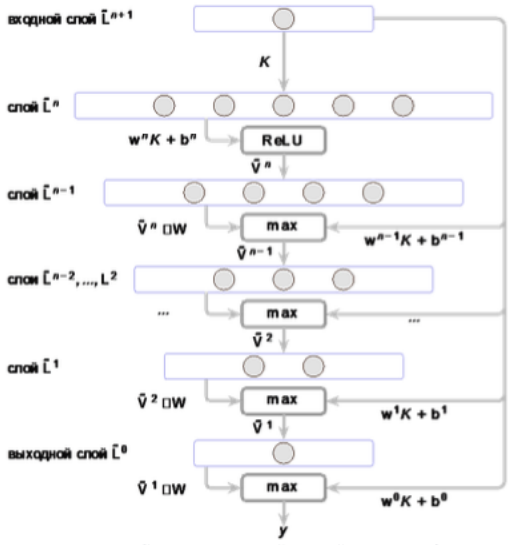

Рисунок 2. Архитектура нейронной сети для оценки американского опциона пут

3.4. Интерпретация весов и обучение

При описанной инициализации сеть в точности воспроизводит дерево CRR глубины *n* в смысле [1]; далее градиентный спуск **изменяет** веса так, чтобы минимизировать отклонение от **выбранных целей** (BS [3] для европейского

эксперимента или густое биномиальное дерево для американского). Интерпретация сохраняется: слои остаются увязанными с экономическим смыслом дерева, но численные значения уточняются данными.

3.5. Два численных эталона для американского пута (концепция)

В проекте используются **два независимых** пути получения биномиальных цен: (1) класс MyIBT_CRR с методом eval_american_put после build(...) — согласован с тем же n, что и сеть, для графиков «CRR vs NN»; (2) функция american_put_prices_binomial(S0, K, T, r, sigma, n=100) — компактная реализация обратного хода в чистом NumPy с $d = 1/u$ и явным риск-нейтральным p, служащая **густым** эталоном для обучения и сравнения. Оба подхода опираются на дискретную логику CRR [2], [4], но реализованы в разных модулях кода, что упрощает проверку согласованности результатов.

3.6. Стек технологий

- **Python 3** — объединение моделей, данных и визуализации.
- **PyTorch** [11] — nn.Module, nn.Linear, nn.Conv1d, torch.optim.Adam [14], nn.MSELoss, режимы train/eval, отключение градиентов в predict.
- **NumPy** — предвычисление u, d, W, выборки страйков, эталон american_put_prices_binomial.
- **SciPy** [12] — scipy.stats.norm.cdf в bs_put_price для формулы Блэка–Шоулза [3].
- **Matplotlib** [13] — построение кривых цен и ошибок.
- **tqdm** — индикатор прогресса по эпохам в train_BTNet.

Фиксация версий библиотек — в **environment.yml** и **requirements.txt** в корне репозитория.

4. Реализация

4.1. Носитель кода и порядок ячеек

Вся логика собрана в одном файле **btnn-bs-torch.ipynb** (корень репозитория). Типичный порядок работы: импорты (numpy, matplotlib.pyplot, tqdm, torch, torch.nn, scipy.stats.norm) → определение эталонного дерева MyIBT_CRR → базовые слои ConvLayer, DenseLayer → BTNetEuropean → MaxoutLayer, BTNetAmerican → train_BTNet → эталоны bs_put_price, american_put_prices_binomial → функции графиков → блок параметров (S_0, T, r, σ, n_dim, диапазон K) → построение crr_tree.build(...) → обучение и тесты.

4.2. Класс MyIBT_CRR

Класс инкапсулирует **имплицитное биномиальное дерево** (в коде — массивы цен узлов и вспомогательных величин). Метод build(S0, t0, T, r, sigma, teta) заполняет сетку; eval_european_put(K) и eval_american_put(K) возвращают цену пута для заданного страйка. Это основной **внутренний эталон** с тем же числом шагов, что у BTNet, при визуализации «зелёной» кривой CRR в plot_comparison.

4.3. Вспомогательные слои

ConvLayer: обёртка над nn.Conv1d(in_channels=1, out_channels=1, kernel_size=2, bias=False). В forward вход unsqueeze(1) → свёртка → squeeze(1), чтобы согласовать ожидаемую PyTorch размерность (N, C, L) с одномерным вектором ценностей.

DenseLayer: nn.Linear с ручной подстановкой weight и bias из NumPy-массивов (через torch.no_grad()), опционально композиция с переданной функцией активации (в проекте — torch.relu для первого слоя).

MaxoutLayer: два параллельных пути — тот же Conv1d(1,1,kernel_size=2) и Linear(1, output_dim) от *K*; выход — torch.max по координатам.

4.4. Модели BTNetEuropean и BTNetAmerican

Конструкторы принимают (n_dim, S0, sig, T, t0, r=None); при r is None подставляется 0,05 — значение по умолчанию в коде. Метод **predict(K)** переводит вход в torch.Tensor, приводит к форме (*N*, 1), вызывает forward в режиме eval() с torch.no_grad() и возвращает **NumPy**-массив предсказаний — удобный интерфейс для сравнения с bs_put_price и массивами CRR без лишнего кода в ноутбуке.

4.5. Функция train_BTNet

Сигнатура: train_BTNet(model, K_train, prices_train, epochs=200, lr=0.01). Реализован **полносводный** градиентный шаг: весь тензор K_train и целевые цены prices_train прогоняются через модель за одну итерацию эпохи (фактически один большой батч размера 500 в стандартном эксперименте). Оптимизатор — torch.optim.Adam(model.parameters(), lr=lr) [14]; функция потерь — nn.MSELoss(). История loss_history накапливается для plot_training_losses. Каждые 50 эпох печатается текущее значение loss — это же можно цитировать в отчёте как лог сходимости.

4.6. Аналитический и биномиальный эталоны

bs_put_price(S, K, T, r, sigma) — векторизованная формула европейского пута [3] через d_1, d_2 и norm.cdf [12]; используется для генерации prices_euro при обучении BTNetEuropean.

american_put_prices_binomial(..., n=100) — отдельная процедура обратного хода в NumPy с выбором max(внутренняя стоимость, дисконтированное ожидание) на каждом шаге; служит целевым значением при обучении BTNetAmerican на выборке K_samples.

4.7. Визуализация

- **plot_comparison** — три кривые (в подписях: Black-Scholes, BTNet, Cox-Ross-Rubinstein), сетка и легенда [13].
- **plot_errors** — две панели: ошибки относительно BS и «NN − CRR» с заливкой области под кривой на нижнем графике.
- **plot_training_losses** — динамика MSE для европейской и американской моделей на одной фигуре.

4.8. Зависимости и документация

Перечень пакетов задан в **environment.yml** (conda, в т.ч. jupyter, pytorch по каналам) и **requirements.txt** (pip). Файл **README.md** описывает назначение репозитория, структуру каталогов, установку окружения и пример использования моделей.

4.9. torchinfo и экспорт ONNX

В отдельных ячейках ноутбука подключается **torchinfo.summary(model, input_size=(1, 1))**, что выводит **иерархию слоёв**, число обучаемых

параметров и формы активаций (для BTNetEuropean при $n = 9$ суммарно 22 параметра у свёрточного блока и т.д. — фактические числа берутся из вывода ноутбука). Это прямое доказательство соответствия реализации требованиям к документированию архитектуры.

5. Тестирование и апробация

План эксперимента. Фиксированные параметры (в духе [1]): $S_0 = 0{,}5$, $t_0 = 0$, $T = 1$, $r = 0{,}05$, $\sigma = 0{,}25$, $n = 9$ для сети и дерева CRR; страйки $K \in [0{,}25; 0{,}75]$; обучающая выборка — 500 случайных K; тестовая — равномерная сетка (например, 51 точка). Европейский случай: целевые цены — по Блэку–Шоулзу [3]. Американский случай: целевые цены — биномиальная модель с $n = 100$ [2], [4]. Обучение: 200 эпох, Adam [14], learning rate 0,01, MSE.

Метрики. На тестовой сетке вычисляются, в частности, **MAE** (средняя абсолютная ошибка) и при необходимости **RMSE** между предсказанием сети и эталоном CRR с тем же n, а также анализируются отклонения от BS [3] и от густого дерева.

Таблица 1. Параметры эксперимента.

Параметр	Значение
S_0	0,5
T	1
r	0,05
σ	0,25
n (сеть / CRR)	9
n (амер. эталон)	100
Эпохи	200
Оптимизатор	Adam, lr = 0,01
Функция потерь	MSE

Таблица 2. Порядок величин ошибки (по результатам эксперимента в ноутбуке; уточнить числа после пересчёта).

Модель / сравнение	Порядок MAE на тестовой сетке K
BTNetEuropean vs CRR (n = 9)	$\sim 10^{-5}$
BTNetAmerican vs CRR (n = 9)	$\sim 10^{-5}$
CRR n = 9 vs BS (европейский)	$\sim 10^{-2}$ (систематическое смещение дискретизации)

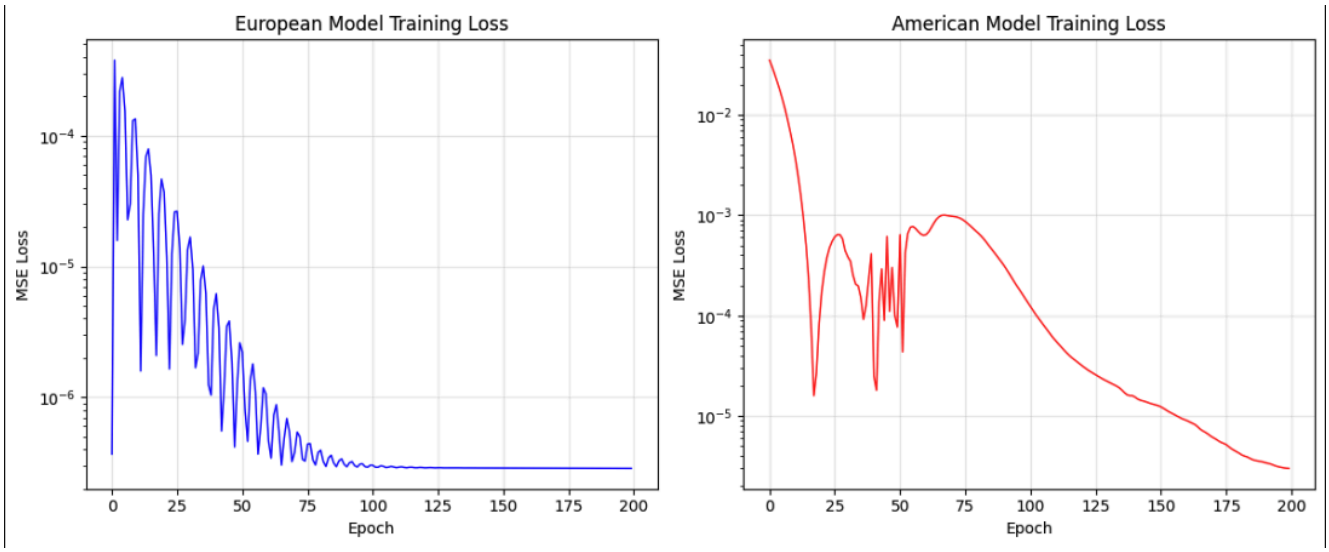

Рисунок 3. Динамика функции потерь при обучении нейронных сетей.

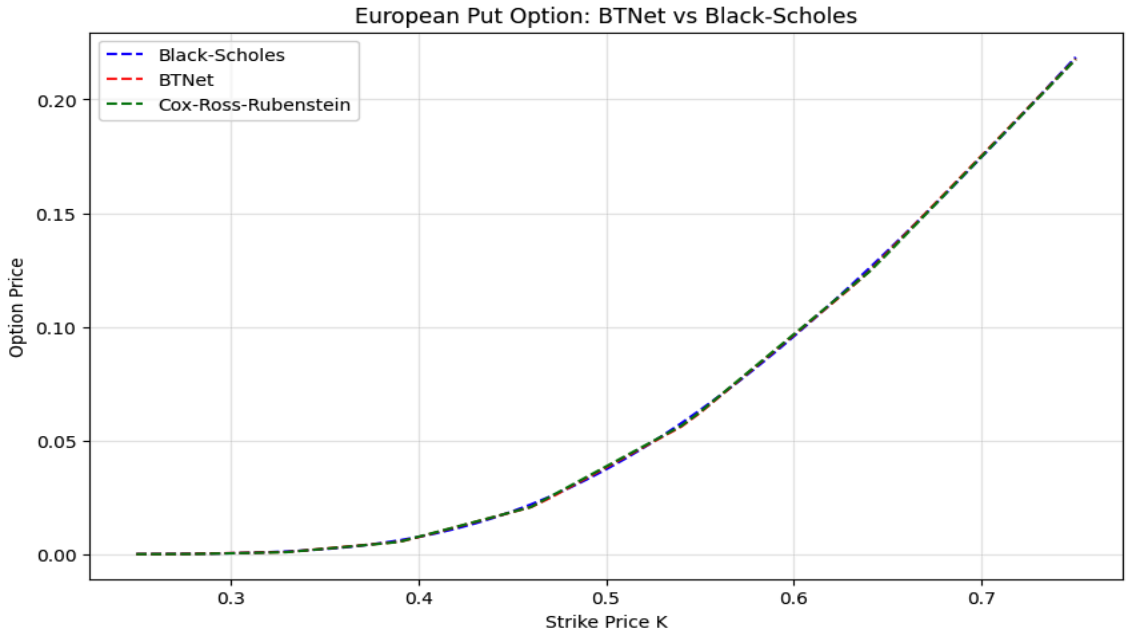

Рисунок 4. Сравнение цен европейских опционов пут в моделях Блэка-Шоулза[3], CRR[2] и нейросетевой модели[1].

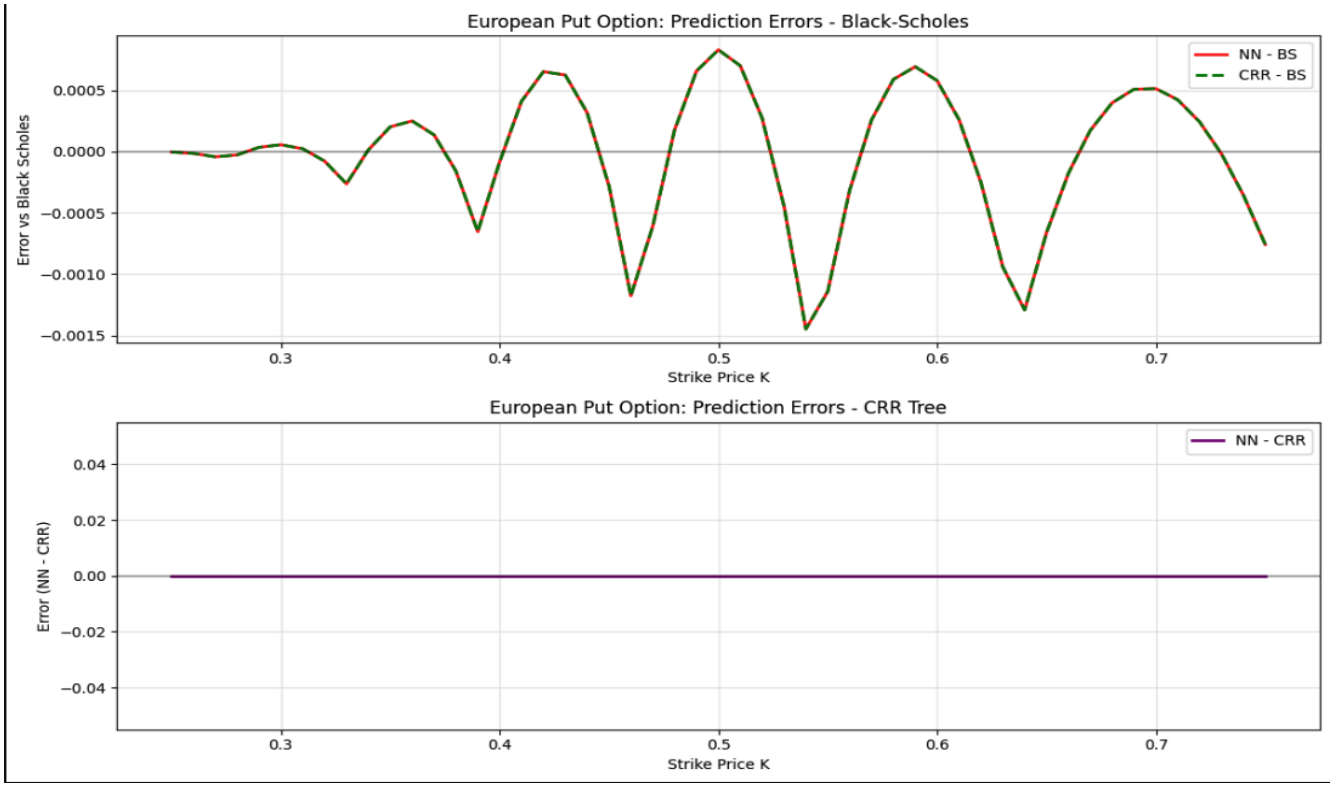

Рисунок 5. Отклонения предсказаний нейросети и модели CRR от модели Блэка-Шоулза[3].

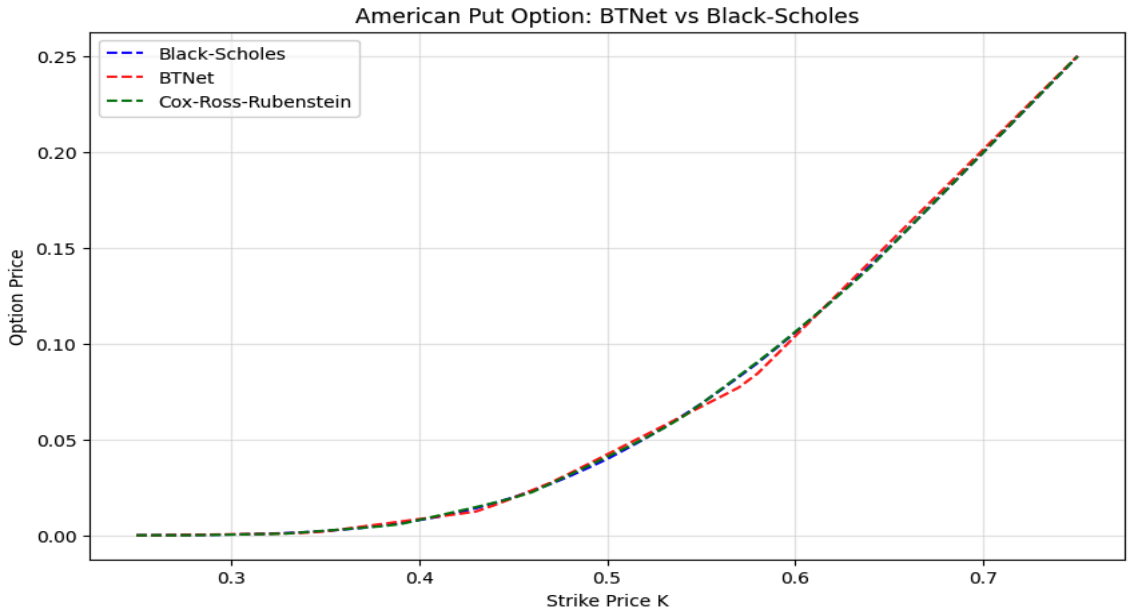

Рисунок 6. Сравнение цен американских опционов пут.

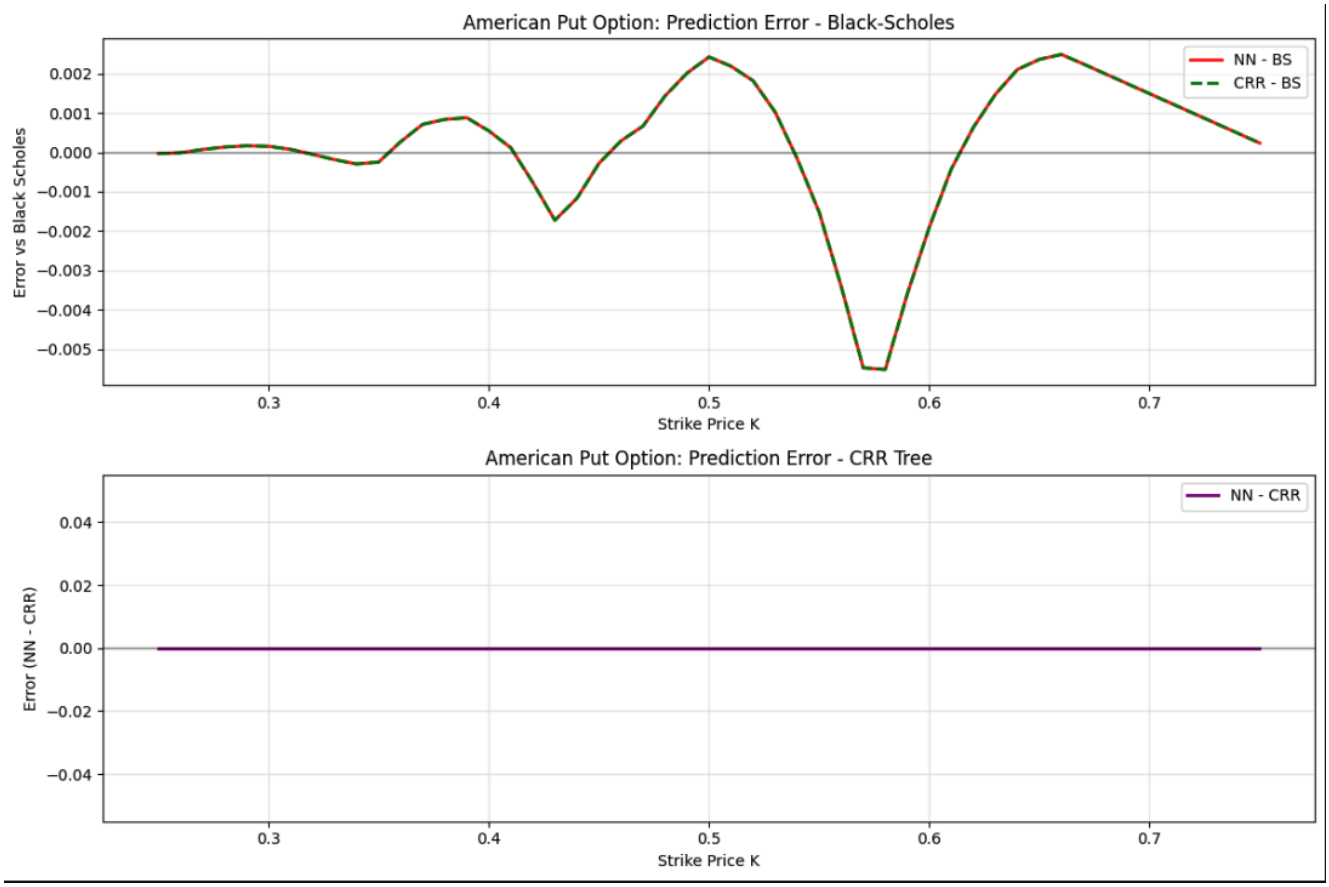

Рисунок 7. Отклонения предсказаний нейросети и модели CRR от эталонной биномиальной модели.

Верификация и сравнение с аналогами. Установлено согласование предсказаний сети с **биномиальным эталоном той же глубины** [2]: сеть воспроизводит именно дискретную модель CRR, а не формулу Блэка–Шоулза [3], если обучать на BS-целях при фиксированной архитектуре $n = 9$ — в согласии с интерпретацией [1]. Отличия от BS и от густого дерева отражают **конечное *n*** и природу американского контракта [4], [8].

6. Заключение

В работе реализованы нейросетевые архитектуры **BTNetEuropean** и **BTNetAmerican**, соответствующие постановке [1] и модели CRR [2]. Проведена апробация на синтетических данных с эталонами Блэка–Шоулза [3] и биномиальных деревьев [2], [4], [5]. Результаты подтверждают **работоспособность** структурированной архитектуры и её способность **аппроксимировать** заданные целевые кривые при малой ошибке относительно CRR с тем же числом шагов.

Практическая значимость — воспроизводимый программный комплекс (Python, PyTorch [11]), пригодный для учебных и исследовательских экспериментов и для демонстрации связи глубоких сетей с классическими численными схемами.

Направления развития: обучение на **рыночных** данных; усложнение модели актива (локальная или стохастическая волатильность) [4], [8]; интеграция эталонов **QuantLib** [10] в тот же конвейер сравнения; перенос критичных блоков из ноутбука в пакет с **CI** и регрессионными тестами.

7. Литература

1. Шорохов С.Г. Оценка финансовых деривативов нейронной сетью на основе биномиального дерева // International Journal of Open Information Technologies. — 2024. — Т. 12, № 5. — С. 108–114.

2. Cox J.C., Ross S.A., Rubinstein M. Option pricing: A simplified approach // Journal of Financial Economics. — 1979. — Vol. 7, No. 3. — P. 229–263.

3. Black F., Scholes M. The pricing of options and corporate liabilities // Journal of Political Economy. — 1973. — Vol. 81, No. 3. — P. 637–654.

4. Hull J.C. Options, Futures, and Other Derivatives. — 11th ed. — Pearson, 2021.

5. van der Hoek J., Elliott R.J. Binomial Models in Finance. — New York : Springer, 2006. — 306 p.

6. Kwok Y.K. Mathematical Models of Financial Derivatives. — 2nd ed. — Berlin : Springer, 2008. — 530 p.

7. Лю С., Оостерли К.В., Ботэ С.М. Оценка опционов и вычисление подразумеваемой волатильности с помощью нейронных сетей = Pricing Options and Computing Implied Volatilities using Neural Networks // Risks. — 2019. — Vol. 7, No. 1. — P. 16. — DOI: 10.3390/risks7010016.

8. Glasserman P. Monte Carlo Methods in Financial Engineering. — New York : Springer, 2003. — XIII, 596 p.

9. Longstaff F.A., Schwartz E.S. Valuing American Options by Simulation: A Simple Least-Squares Approach // The Review of Financial Studies. — 2001. — Vol. 14, No. 1. — P. 113–147.

10. QuantLib Python Documentation : документация. — URL: https://quantlib-python-docs.readthedocs.io

11. Paszke A. и др. PyTorch: An Imperative Style, High-Performance Deep Learning Library // Advances in Neural Information Processing Systems 32. — Curran Associates, Inc., 2019. — P. 8024–8035.

12. Virtanen P. и др. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python // Nature Methods. — 2020. — Vol. 17. — P. 261–272. — DOI: 10.1038/s41592-019-0686-2.

13. Hunter J.D. Matplotlib: A 2D Graphics Environment // Computing in Science & Engineering. — 2007. — Vol. 9, No. 3. — P. 90–95.

14. Kingma D.P., Ba J. Adam: A Method for Stochastic Optimization // Proc. of the 3rd International Conference on Learning Representations (ICLR). — 2015.